



UNIVERSITÉ DE
SHERBROOKE

Normes de codification en C++

GIN950 – Système de surveillance à distance (sans fil) des signes vitaux des nouveau-nés

Révision – 0.1

Table des matières

Table des matières.....	2
Révisions	3
Introduction	4
Format du document	4
Le code.....	4
Général	4
Les noms	5
Les méthodes et fonctions.....	7
Les assertions et la programmation par contrat.....	7
La gestion des erreurs et exceptions.....	8
Les commentaires.....	8
L'indentation du code.....	8
Présentation des fichiers	8
Annexe I – Exemples de code.....	9
Déclaration de variables	9
Structure if	9
Structure for	9
Annexe II – Exemple de commentaire.....	10
Commentaires de fichier	10
Commentaires de classe	10
Annexe III – Exemple de fichier de déclaration	11
Annexe IV – Résumé des règles	12
Annexe V – Résumé des recommandations.....	13

Révisions

RÉVISIONS			
N°	Date	Section / Page	Sommaire de la modification
0.1	3/02/2004	Tout	Version préliminaire, inspirée de la version 1.0 des normes de Copernic Technologies Inc. (Jean-François Roy)

Introduction

Ce document explique les normes pour l'écriture du code C++ dans le cadre du cours GIN950 – Système de surveillance à distance (sans fil) des signes vitaux des nouveau-nés. Le but de ces normes est d'uniformiser le plus possible le code et ainsi faciliter la lecture du code par les autres membres de l'équipe. De plus, en ayant une façon plus standard d'écrire du code, les problèmes seront diminués et la maintenance facilitée.

Le but du document n'est pas d'être très restrictif, mais plutôt le moins restrictif possible. Toutefois, il y va de chaque développeur d'écrire du code lisible et intelligent.

Le code C (dans les cas où le compilateur ne supporterait pas le C++) devra s'inspirer de ces normes, mais il est entendu que certaines règles ne pourront s'appliquer, dû aux limitations du langage.

Format du document

Le document est composé de **règles** et de **recommandations**.

Une **règle** est une directive à suivre obligatoirement, à moins d'une exception majeure. Lors des revues de code, les **règles** seront toujours vérifiées.

Une **recommandation** pour sa part, est une pratique suggérée lors de l'écriture de votre code, mais qui n'est pas obligatoire. Cette pratique se veut souvent le gros bon sens, mais peut ne pas s'appliquer dans certain cas. Il est possible de promouvoir une recommandation en règle pour certains projets.

Le code

Cette section décrit les différentes **règles** et **recommandations** à suivre lors de l'écriture de code en C++.

Général

- | | |
|-------------------------|---|
| Règle 1 | Le code doit être écrit en Anglais . Ce qui implique les noms de classes, méthodes, fonctions, attributs, les commentaires, ... |
| Règle 2 | Toujours mettre une branche default à une instruction switch . Si tous les cas sont sensés être traités, mettre un <code>assert(false)</code> . |
| Règle 3 | Initialiser chaque variable aussitôt déclarée.
<code>int myInt = 0;</code> |
| Règle 4 | Utiliser les formes de cast C++ car elles sont plus faciles à identifier dans le code et indique clairement l'intention du développeur.
<code>static_cast<>, const_cast<>, reinterpret_cast<>, dynamic_cast<>.</code> |
| Recommandation 1 | L'utilisation de constante numérique (magic number) est déconseillée . Utilisez plutôt une variable <code>const</code> identifiant clairement ce que signifie ce nombre. |
| Recommandation 2 | Les lignes de code ne devraient pas être plus de 80 caractères de long . |

- Recommandation 3** Utilisez des **expressions logiques dans les énoncés conditionnels**.
`if (MyFunction() != 0) // OUI`
`if (MyFunction()) // NON, sauf si ça retourne bool.`
- Recommandation 4** Les **résultats d'algorithmes complexes devraient être validés** en DEBUG à l'aide d'un autre algorithme.

Les noms

Cette section explique comment nommer les différentes parties que comporte ou peut comporter un programme C++.

- Règle 5** Les **noms** doivent être **représentatifs** du contexte auquel les entités réfèrent.
- Règle 6** Les noms de **classes, structures, types, méthodes** et de **fonctions** sont formés d'un ou plusieurs mots **commençant par une majuscule suivi de minuscules**.

De plus, pour éviter les conflits et pour facilement retrouver la provenance d'un mot, les **namespaces** doivent être utilisés.

```
namespace ECG {  
    typedef int MyType;  
}
```

```
namespace ECGMem {  
    class ThreadPool;  
    void MyFunction()  
}
```

- Recommandation 5** L'utilisation de **préfixe** est souhaitée pour diminuer les **ambiguïtés**.

- Recommandation 6** Utilisez des **typedef** pour définir un type template.

```
vector<pair<int, double> > v_Vector; // NON  
typedef vector<pair<int, double> > MyVector // OUI  
MyVector v_Vector;
```

- Règle 7** Les **attributs** de classes sont formés d'un ou plusieurs mots **commençant par une majuscule préfixée de 'm_'**.

```
bool m_MyBool;  
size_t m_Size;
```

- Règle 8** Les **variables globales** sont formées d'un ou plusieurs mots commençant par une majuscule préfixée de 'g_'.
- ```
size_t g_Size = 0; // Variable globale
```
- Règle 9** Les noms de **variables** et les **attributs** de structure sont formés d'un ou plusieurs mots. Le premier mot **commençant par une minuscule et les autres mots par des majuscules**.
- ```
bool returnValue;
size_t counter;
```
- Règle 10** Les **paramètres** de fonctions et méthodes sont **préfixés de 'p_'**.
- ```
void MyClass::MyFunction(int p_MyParameter)
```
- Règle 11** Les **attributs et variables pointeurs débutent par la lettre 'p'** pour chaque niveau d'indirection.
- ```
char *pString; // Simple variable.
char **ppString; // Simple variable.
char *m_pMyString; // Membre d'une classe.
void MyFunction(char *p_pMyString);
```
- Règle 12** Les **attributs et variables références non constantes débutent par la lettre 'r'**.
- ```
char &rString = myString; // Simple variable.
char &m_rMyString; // Membre d'une classe.
void MyFunction(char &p_rMyString, const char &p_Password);
```
- Recommandation 7** Les **attributs et variables de type vecteur ou liste** sont préfixés de 'v' ou 'l'.
- ```
vector<int> vIndex; // Vecteur d'index.
vector<int> m_vIndex; // Membre d'une classe.
vector<int>* m_vpIndex; // Pointeur sur vecteur d'index.
vector<int*> m_pvIndex; // Vecteur de pointeur d'index.
```
- Règle 13** Les **constantes** et valeurs définies dans les **énumérations** sont toujours **définies en lettres majuscules, en séparant les mots avec un '_'**.
- Recommandation 8** **Utilisez les énumérations** plutôt que des **constantes individuelles** lorsqu'elles réfèrent toutes à la même notion.
- Recommandation 9** **Favorisez les constantes** plutôt que les macros. Si vous utilisez des macros, faites ceci:
- ```
#define MY_MACRO ((double)1)
```
- Et non ceci:
- ```
#define MY_MACRO 1
```
- Règle 14** Les **fichiers d'inclusion** ont l'extension '.h'. Les **fichiers contenant du code C++** ont l'extension '.cpp' ou '.hpp'. Les méthodes *inlines* doivent être définies dans le fichier '.hpp' et non dans le fichier '.h'. Le fichier '.hpp' est inclus par le '.h'. Les **fichiers contenant uniquement du code C** ont l'extension '.c'.

Recommandation 10 Les **abréviations** devraient être utilisées le moins possible, sauf s'il n'y a pas d'ambiguïté ou qu'une liste définissant chaque abréviation existe dans le projet.

Les méthodes et fonctions

Règle 15 Le **nom des paramètres** d'une méthode ou d'une fonction doivent être définis dans le **'h'**.

```
void MyFunction(int, int = 0); // NON !!!
void MyFunction(int p_Counter, int p_Parameter = 0); // OUI !!!
```

Règle 16 Les **méthodes et opérateurs générés** par le compilateur doivent être déclarés privé si non-utilisé. Si les méthodes générées sont valides, un commentaire doit indiquer que c'est le cas.

Recommandation 11 Les **paramètres par défaut** devraient être définis dans le **commentaire de la méthode ou de la fonction**. De plus, il est souhaitable de laisser la valeur par défaut en commentaire dans la définition de la fonction.

```
// p_MyPrm - This parameter is for... [default = 10]
void MyClass::MyFunction(int p_Counter, int p_MyPrm /* =10 */)
{
    // ...
}
```

Recommandation 12 Les méthodes et fonctions ne devraient posséder qu'un **seul point de sortie (return)**.

Recommandation 13 Les **méthodes d'accès aux attributs d'une classe (lecture & écriture)** doivent être **inline**, définies dans le **'hpp'** et porter le nom de l'attribut.

```
class MyClass {
public:
    int         MyAttribute() const;        // read
    void        MyAttribute(int p_NewValue); // write
private:
    int         m_MyAttribute;
};
```

Les assertions et la programmation par contrat

Règle 17 Les **assertions** doivent être utilisées pour **valider les conditions invalides, mais surtout impossible**. Si une condition dépend de l'entrée d'un usager ou de quelque chose hors de notre contrôle, elle doit être validée dans le code standard et faire partie du contrôle des erreurs du logiciel.

La gestion des erreurs et exceptions

Règle 18 La **gestion des erreurs** est faite à l'aide des **exceptions C++**. Les exceptions doivent toutefois être utilisées avec précaution. Elles **servent à traiter les cas d'erreurs graves**, dans la majorité des cas, il est possible de **retourner une valeur indiquant l'erreur**.

Les commentaires

Règle 19 Toute fonction ou méthode doit obligatoirement avoir un **commentaire** expliquant son but ainsi que ses **paramètres d'entrés et de sorties**.

Règle 20 Ceci dans le but de se repérer plus facilement dans le code. Les **éléments de complexités, optimisation, ...** doivent obligatoirement **être documentés**.

L'indentation du code

Règle 21 Utilisez des **indentations de 4 caractères**. **N'utilisez pas le caractère TAB** car dans certains environnement, le TAB vaut 8 espaces et n'est pas configurable.

Recommandation 14 **Toujours utiliser des accolades pour les blocs de contrôle** afin d'éviter les erreurs et faciliter la modification du code.

```
if (cond != 0)           // NON
    statement
if (cond != 0) {         // OUI
    statement
}
```

Présentation des fichiers

Règle 22 Chaque fichier doit obligatoirement avoir:

- **En-tête du projet;**
- **Notice de copyright.**

Règle 23 L'**inclusion conditionnelle** doit être utilisé dans les fichiers **'h'**. Le nom du *define* à utiliser est le nom du fichier, en majuscule, en remplaçant le **'.'** par un **'_'**.

```
Fichier CGLLib.h
#ifndef CGLLIB_H
#define CGLLIB_H

// Header file code.

#endif // CGLLIB_H
```

Annexe I – Exemples de code

Dans cette section, vous trouverez des exemples qui fournissent un format général à utiliser dans la création de code C++.

Déclaration de variables

Toujours déclarées juste avant leur utilisation, et non au début d'une fonction (par exemple), le plus souvent possible initialisée dès leur déclaration. Un espace avant et après l'opérateur =. Pour les pointeurs (ou les références), l'étoile (*) ou &) est tout de suite après le type de la variable, suivi d'un espace.

```
int var = 0;
MyClass* pVar = NULL;
```

Structure if

Un espace après le *if*, un espace avant et après les opérateurs logiques (==, ||, &&, etc.), un espace après la parenthèse et le premier crochet sur la même ligne. Les lignes consécutives sont tabulées de 4 espaces (pas de *tab*).

```
if (var == otherVar) {
    ...
}
```

Les *if* sans crochets sont interdits, tout comme l'opérateur conditionnel (?:) puisqu'ils ne font que rendre le code moins lisible.

```
if (var == otherVar) doSomething(); // Interdit
if (var == otherVar) // Interdit
    doSomething();
(var == otherVar) ? doSomething() : doSomethingElse(); // Interdit
```

Structure for

Un espace après le *for* et après chaque point-virgule, un espace avant et après les opérateurs (=, <, etc.), un après la parenthèse et le crochet sur la même ligne. L'opérateur de préincrément doit être utilisé. Les boucles *for* sans crochet sont interdites, comme les *if*.

```
for (int i = 0; i < limit; ++i) {
    ...
}
```

Ces exemples illustrent bien la majorité des règles qui doivent être suivies. Pour le reste, il est laissé au programmeur d'utiliser son bon jugement pour créer du code lisible et compréhensible par les autres membres de l'équipe.

Annexe II – Exemple de commentaire

Dans cette section, vous trouverez un exemple de commentaire pour la majeure partie d'un programme.

Commentaires de fichier

Les commentaires suivants doivent se retrouver dans tous les fichiers (en-tête du projet et notice de copyright).

```
//*****
// GIN950 - Système de surveillance à distance (sans fil) des signes vitaux
//          des nouveau-nés
//*****
//*****
// Copyright (c) 2004, Jean-Francois Roy, Eric Lapointe, Francis
// Leblanc-Lebeau, Simon Arpin, Nicolas Beauchamp, Benoit Adam, Ariane
// Authier.
//*****
```

Commentaires de classe

Les commentaires de classes incluent une explication de ce que la classe fait ainsi qu'une description de ses attributs.

```
//*****
// Virtual class to encapsulate thread mecanism.
// CANNOT BE INSTANTIATED. Use function CreateThread.
//*****
class dll_export Thread
{
public:
protected:
    // Group=Private Attributes
private:
    int                m_MyAttr; // This attribute is for...

    // This is a comment that is too long to go after the attribute,
    // so we simply put it before the attribute
    long               m_MyLong;
    // Group=
...
};
```

Annexe III – Exemple de fichier de déclaration

```
//*****
// GIN950 - Système de surveillance à distance (sans fil) des signes vitaux
//      des nouveau-nés
//*****

//*****
// Copyright (c) 2004, Jean-Francois Roy, Eric Lapointe, Francis
// Leblanc-Lebeau, Simon Arpin, Nicolas Beauchamp, Benoit Adam, Ariane
// Authier.
//*****

#ifndef ECGTYPES_H
#define ECGTYPES_H

#ifndef ECG_H
#   include "ECG.h"
#endif

namespace ECG {

class CriticalSection;
class DebugStream;
class EventSemaphore;
class RealStopWatch;
class CPUStopWatch;
class StopWatch;
class ThisThread;
class Thread;
class ThreadPool;
class UnitTest;

} // namespace ECG

#endif // ECGTYPES_H
```

Annexe IV – Résumé des règles

Règle 1	Le code doit être écrit en Anglais ; classes, méthodes, fonctions, attributs, les commentaires, ...
Règle 2	Toujours mettre une branche default à une instruction switch .
Règle 3	Initialiser chaque variable aussitôt déclarée.
Règle 4	Utiliser les formes de cast C++ car elles sont plus faciles à identifier dans le code et indique clairement l'intention du développeur.
Règle 5	Les noms doivent être représentatifs du contexte auquel les entités réfèrent.
Règle 6	Les noms de classes, structures, types, méthodes et de fonctions sont formés d'un ou plusieurs mots commençant par une majuscule suivi de minuscules . De plus, pour éviter les conflits et pour facilement retrouver la provenance d'un mot, les namespaces doivent être utilisés.
Règle 7	Les attributs de classes sont formés d'un ou plusieurs mots commençant par une majuscule préfixée de 'm_' .
Règle 8	Les variables globales sont formées d'un ou plusieurs mots commençant par une majuscule préfixée de 'g_' .
Règle 9	Les noms de variables et les attributs de structure sont formés d'un ou plusieurs mots. Le premier mot commençant par une minuscule et les autres mots par des majuscules .
Règle 10	Les paramètres de fonctions et méthodes sont préfixés de 'p_' .
Règle 11	Les attributs et variables pointeurs débutent par la lettre 'p' pour chaque niveau d'indirection.
Règle 12	Les attributs et variables références non constantes débutent par la lettre 'r' .
Règle 13	Les constantes et valeurs définies dans les énumérations sont toujours définies en lettres majuscules, en séparant les mots avec un '_' .

Règle 14	Les fichiers d'inclusion ont l'extension 'h' . Les fichiers contenant du code C++ ont l'extension 'cpp' ou 'hpp' . Les méthodes <i>inlines</i> doivent être définies dans le fichier 'hpp' et non dans le fichier 'h' . Le fichier 'hpp' est inclus par le 'h' . Les fichiers contenant uniquement du code C ont l'extension 'c' .
Règle 15	Le nom des paramètres d'une méthode ou d'une fonction doivent être définis dans le 'h' .
Règle 16	Les méthodes et opérateurs générés par le compilateur doivent être déclarés privé si non-utilisé.
Règle 17	Les assertions doivent être utilisées pour valider les conditions invalides, mais surtout impossible .
Règle 18	La gestion des erreurs est faite à l'aide des exceptions C++ .
Règle 19	Toute fonction ou méthode doit obligatoirement avoir un commentaire expliquant son but ainsi que ses paramètres d'entrées et de sorties .
Règle 20	Les éléments de complexités, optimisation, ... doivent obligatoirement être documentés .
Règle 21	Utilisez des indentations de 4 caractères . N'utilisez pas le caractère TAB .
Règle 22	Chaque fichier doit obligatoirement avoir: • En-tête du projet; • Notice de copyright.
Règle 23	L' inclusion conditionnelle doit être utilisé dans les fichiers 'h' . Le nom du define à utiliser est le nom du fichier, en majuscule, en remplaçant le '.' par un '_' .

Annexe V – Résumé des recommandations

Recommandation 1	Les lignes de code ne devraient pas être plus de 80 caractères de long .
Recommandation 2	Utilisez 0 au lieu de NULL pour comparer des pointeurs .
Recommandation 3	Utilisez des expressions logiques dans les énoncés conditionnels .
Recommandation 4	Les résultats d'algorithmes complexe devraient être validés en DEBUG à l'aide d'un autre algorithme.
Recommandation 5	L'utilisation de préfixe est souhaitée pour diminuer les ambiguïtés.
Recommandation 6	Utilisez des typedef pour définir un type template.
Recommandation 7	Les attributs et variables de type vecteur ou liste sont préfixés de 'v' ou 'l'.
Recommandation 8	Utilisez les énumérations plutôt que des constantes individuelles lorsqu'elles réfèrent toutes à la même notion.
Recommandation 9	Favorisez les constantes plutôt que les macros. Si vous utilisez des macros, faites ceci: #define MY_MACRO ((double)1)
Recommandation 10	Les abréviations devraient être utilisées le moins possible.
Recommandation 11	Les paramètres par défaut devraient être définis dans le commentaire de la méthode ou de la fonction .
Recommandation 12	Les méthodes et fonctions ne devraient posséder qu'un seul point de sortie (return) .
Recommandation 13	Les méthodes d'accès aux attributs d'une classes (lecture & écriture) doivent être inline, définies dans le '.hpp' et porter le nom de l'attribut.
Recommandation 14	Toujours utiliser des accolades pour les blocs de contrôle afin éviter les erreurs et faciliter la modification du code.